

atomOS Entertainment Possibility Spaces

Games spatial experiences and sensory entertainment on the Klein log polar substrate

Version 0.1 | 13 September 2026

Engine concept and formalization: Tom Klootwijk

Entertainment concepts and document development: AI-assisted collaboration with Tom Klootwijk

The strongest entertainment direction is a game where players discover, collect and alter the rules of the world they inhabit. A room can contain its own editable rulebook. A creature can inherit the program that controls its behavior. A weapon can carry a rule that changes how the next encounter unfolds. The player sees those changes happen and learns to use them.

This overview turns the SHA-256-seeded, self-referential and self-modifying atomOS architecture into concrete entertainment proposals. It covers games, real-world spatial VR/AR/MR, sensory pleasure, biofeedback and hypothetical neural or biochemical interfaces. Every proposal retains the required Klein topology and log-polar lookup table substrate, with operator predicates packed into texture words. Stable identifiers, playable openings and prototype outlines make the record expandable.

Recommended first game: Foldhouse, a puzzle adventure about editing the rules of impossible rooms. Its first prototype can express the central idea with one room, one editable machine and an observable change in behavior. Players can judge whether the experience is enjoyable before a large world, creature population or multiplayer system is built.

The first entertainment branches

ID	Working title	What the player does	Main appeal
E01	Foldhouse	Rewire a room and exploit its reflected connections	Discovery and clever solutions
E02	Seedlings	Raise creatures and breed useful behavior	Attachment and surprising offspring
E03	Rewrite Arena	Fight with weapons that alter combat rules	Skill and combinations
E04	Loop Heist	Program small thieves and redirect the vault	Planning and comic failure
E05	Living Mixtape	Perform with musical creatures that edit their patterns	Expression and musical surprise
E06	Pocket Universes	Trade short worlds and remix their challenges	Social play and creative authorship

Spatial and embodied branches E07-E12 add room-scale play, an impossible carnival, a sensory instrument, physiological feedback, neural input and flavor experiences. H01-H02 record speculative neural and biochemical directions. These are proposals; none is delivered as a playable application by this document. Working titles identify concepts within this overview.

The architecture each concept inherits

The target application reads an encoded description of its own state and rules, proposes a change and continues from an accepted new version. A fixed native executor interprets that changing application data. Self-modification here means editing represented rules or their composition; rewriting CUDA machine instructions is not required. [S1, pp. 25, 30]

Part	Concrete role in the proposed entertainment runtime
SHA-256 genesis and seed	Retain the full identity digest. Use its two 128-bit halves to seed log-radius and angle through the documented projection. Record the generator and configuration versions for any additional generated content.
Klein topology	Use the actual quotient connections and reflected angular seam in simulation. A 2D chart, portal view or 3D presentation can make these connections visible to the player.
Log-polar LUT	Address the operator dictionary in log-radius and angle. In this chart, equal radial steps correspond to multiplicative physical-radius changes when mapped back to radius.
Packed operator words	Compile declared SDF predicates into word masks for texture-backed execution. The current representation stores predicate bits, not every continuous distance value or all application data.
Self-description	Expose a bounded record of the application's rule graph, selected state and current version to its own interpreter.
Checked modification	Stage proposed rule and state changes, verify them, and publish an accepted version between epochs. Any changed LUT is a new immutable snapshot.
Version history	Link canonical event records to the previous ledger head. Retain the records and required content to support replay and forks.

The proposed loop is **seed and configuration -> initial Klein state -> execute packed predicates -> inspect state and rules -> propose an edit -> verify -> commit a new version -> continue**. Each game specifies which observations may trigger edits and which rules are editable.

What exists and what must be added

The current joint construction couples a NOR controller to Klein branching through one immutable packed SDF predicate atlas. Its recorded validation includes 104 native cases and 18 sanitizer runs. Four measured joint cache profiles reached the compulsory cold-miss floor for the complete 1 KiB atlas. These are finite, scoped results. [S2, S3]

The complete SHA-256 seed projection into the joint application loop, a self-description interface, geometry-to-controller feedback and an interpreter that commits its own rule edits remain extensions. Current SHA-256 use includes host provenance; current joint program wiring does not rewrite itself. A scalar SDF for every named original source operator is not established. [S2-S4]

SHA-256 is an identity and integrity mechanism, not reversible compression of an arbitrary game. Distinct identities can occupy one quantized cell. A fixed working budget supports continuing play by recycling state or retaining history elsewhere; it does not retain an unlimited exact history in finite storage. [S1, pp. 8, 30]

E01 Foldhouse

Puzzle adventure | Solo first | Recommended first playable

You explore a house whose rooms contain small machines describing how they work. Doors, lights and moving platforms obey visible rules. You can move a limited number of rule pieces, and some machines can change their own next instruction after an event. The pleasure is understanding a strange room well enough to outsmart it.

The first minute

You enter a room with a locked exit, a pressure plate and a rolling light orb. Walk through an illuminated seam doorway and you return on the reflected side of the room. A floor diagram previews where that crossing leads. The orb misses the plate. Open its two-slot rule panel and change its next turn from left to right. Send it through the seam again; it reaches the plate and opens the exit.

The repeated game loop

Observe a room, predict one consequence, edit a rule, run the room and use the result. Pause and step through an unfamiliar mechanism. Rewind to a retained snapshot when the experiment fails. Later rooms let a machine inspect whether its last route succeeded and choose a different instruction for its next lap. The player decides which changes the machine is allowed to make.

Progression comes from new combinations: a platform that reverses after carrying you, a door that changes its destination after a bell, or two copies of a room that share a starting seed but evolve under different rules. A solved room becomes a small creation the player can retain and remix.

How the substrate becomes gameplay

The Klein seam provides a consistent reflected route. Log-radius and angle provide the room's simulation coordinates; a readable presentation can flatten or remap them. Packed predicates define declared activation regions and gate behavior. A room's editable rule record describes the same mechanism that executes it. Its self-edit is accepted between ticks, then appears in the next run. The root seed identifies the starting recipe; retained revisions identify the player's altered room.

Smallest playable version

Build one fixed-camera room with one orb, one plate, one exit, two clearly marked seam crossings and a two-instruction editor. Include one machine-authored rule change: after visiting the plate, the orb rewrites its next turn. Show the before and after instruction. Support reset and one retained rewind point. This slice needs no generated campaign.

The main fun risk

If seam crossings feel arbitrary or edits have invisible consequences, the puzzle becomes guesswork. Show a destination preview and a short path trace. In a play session, observe whether a new player can predict the second crossing and deliberately solve the room. Expand only when the player can explain the successful idea in ordinary language.

E02 Seedlings

Creature raising and collection | Solo sanctuary with later local exchange

You raise tiny creatures that carry editable behavioral recipes. One follows light, another protects eggs, and a third gathers objects into spirals. Their bodies and behaviors come from declared generators. You guide them through play, select traits for offspring and build a sanctuary that encourages interesting interactions.

The first minute

A seed hatches into a small creature. Place a glowing berry across the reflected seam of its habitat. The creature takes an unexpected route and reaches it. Its simple behavior panel shows what happened: it followed the light, crossed the seam and returned with reversed local orientation. Give it a second berry and choose whether it should repeat that route or inspect a different landmark.

The repeated game loop

Hatch, observe, arrange a habitat, teach through a few explicit choices and enter small challenges. Breed compatible rule fragments to obtain offspring with recognizable family behavior. Keep a favorite ancestor and compare descendants in the same obstacle course. Unlock new toys and habitat mechanisms through caring for creatures and completing challenges.

Self-modification has an understandable trigger: a creature records which of two routes reached food, then proposes changing its preferred route. A later trait can alter how it proposes that change. These are visible, bounded adaptation rules. The game does not need to claim consciousness or unexplained intelligence.

How the substrate becomes gameplay

Each full seed identity remains distinct even if two creatures occupy the same packed cell. Klein transport creates characteristic routes and reunion patterns. Log-polar rings organize the habitat around landmarks. Packed regions define declared sensory or interaction predicates. Each creature has a rule record it can inspect and revise under its trait policy. Parent references and retained version records support family trees and restoring a favorite behavior.

Smallest playable version

Use one habitat, three recognizable appearances, two editable behavioral traits, one adaptation rule and a short berry-fetch challenge. Cap the active population and keep a small nursery archive. Derive appearance from a specified visual generator; the predicate atlas alone does not supply finished creature art or animation.

The main fun risk

If every mutation looks random, players will not form preferences or attachments. Preserve recognizable traits and show one meaningful behavioral change at a time. A successful first session produces a creature the player wants to keep and an offspring whose difference they can explain.

Future branches include obstacle leagues, cooperative habitat building, creature-operated musical toys and expeditions that reward different learned routines. Saved lineages need their actual recipes and records; a digest alone cannot restore a lost creature.

E03 Rewrite Arena

Action arena and short roguelite runs | Solo first

You fight with weapons that carry small, inspectable rule programs. A shot can split after a seam crossing, become a shield after a miss, or change the next shot's route after a successful hit. Between waves, choose a rule fragment and build a weapon combination you can learn to control.

The first minute

Fire at a target behind a barrier. A marked seam route sends the projectile around the arena and back from the reflected side. Hit the target to charge a rule socket. Install a fragment that changes the weapon's next shot into two weaker projectiles after one seam crossing. The preview shows the new route before the next wave begins.

The repeated game loop

Dodge, aim, exploit the seam and earn fragments. Between short waves, compare two possible rule changes. Choose one, then learn the timing of the resulting weapon. Strong combinations trade damage, coverage, resource cost and predictability. A run ends in a boss encounter whose defenses reward understanding the rules you assembled.

The self-editing component belongs to the weapon program. For example, after three misses it proposes replacing its next firing pattern with a narrower burst, then restores its old rule after a hit. The event and edit are visible. Keep these changes within a small vocabulary so combat remains readable at speed.

How the substrate becomes gameplay

Klein transport creates flanking routes and reflected return shots. Log-polar coordinates can support rings of threats and angular attack patterns around an arena focus. Packed predicates supply declared activation, shielding and operator regions. Weapon descriptions are interpreted data that can stage their own revisions. A seed plus the exact initial configuration and retained input sequence supports a replayable run; version hashes identify the particular build and history.

Smallest playable version

Build one arena, one movement model, one enemy type, two base firing rules and one event-triggered rule replacement. Use a fixed number of projectile slots and a short wave. Keep the seam path visible and make weapon changes happen at clear moments. Save one successful run locally for replay under the same declared execution semantics.

The main fun risk

The screen can become unreadable, and an adaptive weapon can undermine the player's muscle memory. Let players preview and lock a rule. Use strict visual limits on splitting and a clear cue when a rule changes.

Responsive controls, collision behavior, effects, sound and frame pacing require game systems beyond the current kernel. The recorded endpoint occupancy operations do not establish swept projectile collision or a complete physics engine. First judge whether aiming through the seam feels satisfying; only then increase rule complexity.

E04 Loop Heist

Programming heist and automation puzzle | Solo or later local cooperation

You direct small thieves through a vault made of repeating corridors, mirrored passages and programmable security machines. Rather than control every step, give a thief a short routine and let it change that routine after it learns something. The entertainment comes from an elegant plan, a ridiculous failure and the next improved attempt.

The first minute

A robot thief reaches a locked gate. Its routine says to follow the wall until it finds a switch. A Klein seam returns it behind the gate with its local orientation reflected. It picks up a key, but its old routine sends it back toward the alarm. Change its rule to return to the exit after acquiring the key. Run the plan again and watch the theft complete.

The repeated game loop

Scout a compact vault, assign a few instructions, run the crew and inspect the outcome. Rewind the plan to a retained start point. Add a branch for a guard, coordinate a distraction, or let a thief replace its search routine after finding the target. Later missions let the crew alter a machine that produces the vault's patrol schedules.

The player earns extra tools by stealing optional objects or completing a mission with fewer edits. Short replays make failures entertaining and invite experimentation. Multiple solutions should be valid when the declared objective and world rules permit them.

How the substrate becomes gameplay

Klein connections make route planning depend on parity and the reflected seam. Log-polar coordinates organize vault rings and radial access. Packed predicates represent declared access zones, triggers and logical machinery. A thief's inspectable program can replace its own search branch with an escape branch. The vault generator can also be represented as editable rules in later missions. Retained hashes distinguish the original challenge from a remixed security layout.

Smallest playable version

Use one thief, one key, one alarm, one seam connection and a four-instruction routine. Give the thief one explicit self-edit after key acquisition. Add run, pause, single-step and reset controls. A complete mission should take only a few attempts and allow at least two intentional routes.

The main fun risk

A large instruction language can turn a heist into homework. Begin with readable actions and show a ghost of the predicted next move. A failed plan should reveal a useful cause, not demand inspection of raw engine records.

Expansion can add cooperative roles, guards with inspectable patrol routines, competing thief crews and a level editor. Cross-player sharing would require a content package and compatibility checks; a shared hash is an identifier, not the level itself.

E05 Living Mixtape

Musical toy and performance game | Solo creation with later local duet play

You arrange musical creatures around a looping stage. They play short phrases, respond to your gestures and can edit the pattern they will play next. You can keep a favorite phrase, breed a variation or make two parts answer each other. It is a playable instrument with visible behavior.

The first minute

Place a drummer and a chiming creature on two rings. Start the pulse. Send the chiming creature through the marked seam and hear the phrase change under a declared musical mapping. Pin the part you like. Tap a response; the drummer detects the selected response event and proposes adding one beat on its next loop. Accept it and hear the duet change.

The repeated game loop

Add a part, perform a gesture, listen and keep or revise the result. Earn new sounds through short musical challenges or explore freely. A phrase can inspect its last few declared events and change its next pattern according to a small rulebook. Players choose the scope of that freedom and can freeze any part before adding another.

The pleasure is hearing an idea become a coherent loop, then discovering an appealing variation. A challenge mode might ask the player to bring three wandering parts into a call-and-response pattern. A free mode would emphasize saved performances and quick remixing.

How the substrate becomes gameplay

Angular phase provides a natural loop position, while log-radius can select a declared tempo or register mapping. Those musical meanings are application choices. Klein parity can select an alternate phrase or reverse a step order under an explicit mapping; the topology does not itself produce an audio effect. Packed predicates define trigger regions and rule conditions. The part's own editable sequence and update rules provide self-reference and self-modification.

Smallest playable version

Use three parts, an eight-step sequencer, one seam interaction and one quantized rule edit at a bar boundary. Offer pin, undo and export of a retained local performance record. A seed initializes the same declared musical recipe; the record must also retain gestures, rule changes, sound assets and relevant versions.

The main fun risk

Unrestricted mutation can produce noise or remove a phrase the player liked. Keep rhythm changes on bar boundaries and let the player constrain scales, density and variation. Audio scheduling, synthesis, sample assets and low-latency output are additional systems; a shader or texture-cache result does not establish audio timing.

The first successful session ends with a short loop the player wants to hear again. Later versions can add audiovisual stages, rhythm challenges and local duets without changing that basic goal.

E06 Pocket Universes

Short challenge collection and creation toy | Solo creation and local pass and play

You make small playable worlds and hand them to someone else as challenges. One asks them to escape a room whose doors change after each bell. Another contains a tiny creature race. A third asks them to catch a light that can rewrite its escape rule. Each world has an inspectable starting recipe and a history of accepted remixes.

The first minute

Open a prepared pocket world. Collect three lights before a wandering creature reaches them. Discover that crossing the seam returns you on the reflected side, giving you a shortcut. At the finish, move one rule piece so that a collected light changes the creature's next route. Reset the world and hand over the controls. The next player encounters your variation.

The repeated game loop

Play a challenge, inspect the rule that made it interesting, change one element and challenge another player. Retain the original and the remix. Compare how the same seed develops under different choices. A later creator mode combines small rule modules into escape rooms, races, rhythm toys and competitive score attacks.

Self-modification can be the rule of a specific pocket world: a creature changes its pursuit routine after losing a race, or a room replaces a door rule after someone uses a shortcut. The creator declares that behavior and can watch the version change. Surprise remains traceable to a rule the player can discover.

How the substrate becomes gameplay

Every world runs on the required Klein/log-polar substrate, with a presentation that makes its useful connections readable. Packed predicates provide the shared operator vocabulary. The world includes data describing its own editable rules and retained state. A genesis seed identifies the initial recipe; parent-linked revisions distinguish remixes. Transfer includes the required content and versions, not merely a hexadecimal digest.

Smallest playable version

Provide one prepared collect-the-lights challenge, one editable rule, a local save slot and a reset button. Allow two people to take turns on the same device. Keep the creator UI to a few choices. Record a retained original and one child version, and show the single rule that differs.

The main fun risk

A creator platform can become an empty toolbox. Start with a game that is enjoyable before editing, then expose one rewarding modification. A successful session produces a remix that the next player can understand and wants to retry.

Broader sharing, content discovery, moderation, synchronization and account services would be separate product work. Local worlds and explicit imports can explore the core entertainment loop first. No service integration or publishing action is part of this overview.

Spatial and embodied entertainment

VR replaces the visible environment; AR overlays digital content on a view of the physical world; MR uses tracked spatial relationships to make digital and physical objects interact. These categories overlap across platforms. For atomOS, the important distinction is what the player sees, what the hardware measures and which outputs the application can actually deliver.

The physical room and the Klein simulation need separate coordinate systems. Anchors associate measured locations with declared chart locations. Virtual portals or transitions express the quotient connections; a seam does not physically bend a room or make a wall traversable. OpenXR documents spatial entities, planes, markers, anchors and persistence as available API capabilities, with support dependent on the runtime. [S6]

Interface readiness and application status

Layer	Evidence level	What an atomOS branch could use
Spatial VR AR MR	Existing interface category	Tracked views and device-supported anchors for a new game integration. [S6]
Audio and vibration	Existing interface category	Authored sound and supported haptic patterns selected by declared events. [S7]
EEG EMG and other biosignals	Existing measurement category	A separately validated decoder or signal feature as an explicit game input. [S8]
Wearable smell and digital taste	Published research prototypes	Custom sensory installations requiring appropriate hardware and evaluation. [S9, S10]
Direct neural sensory feedback	Clinical research evidence for specific tasks	A hypothetical entertainment interface; existing results do not establish arbitrary sensory experience. [S11, S12]
Direct biochemical pleasure control	Speculative application horizon	A research question with no established atomOS body model or validated control mapping.

These levels describe external interface evidence, not completed atomOS integrations. None of the listed sensors, actuators or XR runtimes is connected by the current kernel validation.

A body and room feedback loop

The proposed loop is **tracked room and chosen signals -> explicit calibrated features -> bounded Klein state -> represented experience rules -> checked content update -> compatible display or actuator output -> user response**. Each signal keeps its timestamp and validity. A missing or noisy measurement remains unavailable; it does not become a zero-valued emotion or an automatic control instruction.

The user can choose a desired experience such as playful, soothing, intense or surprising. Adaptation changes a declared content rule within that choice. An explicit rating remains distinct from a physiological measurement. The engine represents and schedules a relationship between input and output; it does not make that relationship biologically true by encoding it in a LUT.

E07 Foldhouse in your room

Mixed reality puzzle adventure | New integration using existing spatial interface categories

Your table becomes a machine, the space above it contains a moving light, and a virtual doorway on a wall reveals the reflected side of a pocket world. You solve puzzles by picking up visible rule pieces and placing them into the machine. The real furniture remains a physical reference throughout the experience.

Opening and loop. Place the world on a detected table, catch a drifting light and send it through a marked virtual portal. Its Klein return appears at a different declared chart location. Move a turn rule with your hand, run the machine and open a virtual compartment. Continue by finding a rule, predicting its spatial effect and using that effect to solve a short room puzzle.

Self-modification. After the light reaches a target, the machine inspects that event and replaces its next route instruction. The change appears on the same panel the player edits. Packed predicates govern the declared virtual interaction regions. The seed supplies the recipe, while the room calibration and anchor records supply physical placement; both matter to replay.

First slice and fun risk. Use one table anchor, one virtual portal, one object and one editable rule. Begin with stationary or teleport-based viewing. The main risk is a convincing visual object whose interaction does not match hand tracking. Clear highlighting and a supported controller fallback preserve the puzzle when tracking is uncertain. Do not assume that a physical prop may be moved or touched solely because a virtual rule permits it.

AR branch. On a compatible phone or glasses, place a pocket creature at a chosen real-world landmark. Find its next anchored patch, collect a rule fragment and change how it searches that patch. The self-edit changes virtual behavior after an observed game event. Klein connections link the virtual patches; ordinary physical navigation connects the real locations. The seed and retained rules define the challenge, while anchor records establish its placement.

E08 Impossible Carnival

VR arcade collection | New game integration using existing display and input categories

Play short booths where a ball circles a Klein arena, a target reappears reflected, and the machine changes one rule between rounds. One booth is seam billiards, another is a creature race, and another is a rhythm tunnel that you conduct with your hands.

Opening and loop. Throw a virtual ball at a visible return portal. Watch it emerge behind a target, then choose a rule that changes what the bumper does after its next hit. Aim again and beat your score. Win a small rule fragment, try a new booth and assemble a favorite local challenge collection.

Substrate and first slice. Log-polar rings organize the booth's simulation; Klein transport supplies the surprising return; packed predicates and mutable rule records drive its logic. Retain a seed, the declared rules and recorded inputs for a challenge replay. Build one seated or standing booth with a single throw mechanic and one between-round self-edit before adding a full park.

The main entertainment risk is disorientation replacing mastery. Keep the player viewpoint stable while virtual objects take unusual routes. Actual headset rendering, tracking, interaction latency and comfort need their own evaluation; cache retention alone establishes none of them.

E09 Sensory Loom

Multisensory performance and pleasure toy | Ordinary audiovisual and haptic prototype first

You weave a moving ribbon of sound, light and touch through a virtual space. Pin a phrase that feels satisfying, stretch its rhythm, send it through a reflected seam and compare the return. The objective can simply be enjoyment: composing an experience you want to repeat, without a score or a victory condition.

A concrete session

Choose a short palette of sound and supported vibration patterns. Draw a loop with a controller or tracked hand. The loom plays it as light and audio, with an explicitly paired tactile phrase. Turn the loop through a Klein crossing to select its reflected variation. Keep the original, keep the variation or alternate them. After several repetitions, the loop proposes one small change to its own sequence, which you can accept, reject or freeze.

Later installation versions could add air movement, purpose-built thermal interfaces or localized scent. Each channel needs its own hardware profile and evaluation. A vibration pattern is not an arbitrary surface texture, and simultaneous output commands do not guarantee simultaneous perceived sensations. Existing haptic SDKs supply one practical interface category; wearable olfactory devices have been demonstrated as research prototypes. [S7, S9]

What pleasure means in this concept

Pleasure is the user's reported enjoyment of a pattern: pleasant touch, musical satisfaction, anticipation, sensory surprise or a sense of agency. The player chooses the palette, intensity preference and duration, and can stop or pin a pattern directly. The system can learn an explicit preference between two variants without claiming to measure a universal pleasure value.

A later local duet mode lets two participants send selected phrases to each other. Each recipient chooses which channels and patterns to accept. The entertainment is a shared performance and an exchange of sensations, with each participant retaining direct control.

The exact substrate role

Log-polar coordinates describe phrase phase and declared variation rings. Klein parity selects a stated transformation such as changing voice assignment or reversing a spatial motif. Packed predicates detect the events that trigger phrase rules. The phrase carries an inspectable description of its own generator and may stage an edit to that generator. The SHA-256 seed and retained revisions identify the content recipe, not a reproducible internal feeling.

Smallest version and expansion decision

Build a seated audiovisual loom with one supported vibration device, three authored phrases and one bounded self-edit at a phrase boundary. Ask which version the participant prefers and whether they can predict the effect of their gesture. Add channels only when they contribute a perceivable, wanted difference. A sensory composition may be repeatable as output data while the participant's experience varies between sessions.

E10 Living Pulse

Body-responsive world and musical game | New biofeedback integration

A small luminous world responds to a chosen body signal. A supported breathing measure can expand a virtual canopy; a measured pulse can drive a visual rhythm; a deliberate muscle gesture can select a musical branch. These are declared mappings, so the player understands what the instrument is listening to.

Opening and loop. Choose one input, see a short live preview and move a physical gesture or ordinary breath through the interface. Watch the canopy answer. Change a rule that controls how the next visible branch responds, then compose a short scene by repeating or varying your input. The generator may inspect its recent event density and simplify its next pattern to preserve a readable performance.

Pleasure and scope. This can be a soothing sensory toy, a rhythm challenge or a playful body instrument. The goal is a chosen experience, not a treatment or a physiological optimization target. A pulse trace is not a direct label for enjoyment, consent or a particular emotion. Missing input pauses its adaptive contribution and leaves explicit user controls available.

First slice. Use one compatible signal source, one on-screen world, one controllable rule and a fixed set of audiovisual outputs. Log-polar phase and Klein transformations shape the generated scene; packed predicates implement its event rules. Record content versions separately from any retained sensor data. The main fun risk is that the mapping feels delayed or unrelated to the player's action.

E11 Neural Instrument

EEG-driven selection or performance experiment | Research integration

The player uses a separately validated EEG decoder to select between two musical or world actions, then hears and sees the result. A successful command might place a sound seed, hold a phrase or choose the next reflected route. The initial fantasy is a new way to play an instrument, not the reading of arbitrary thoughts.

Opening and loop. After device-specific calibration, attempt a supported command and watch an explicit confidence indicator. Confirm or cancel the proposed action with an ordinary control. As the session develops, the instrument can inspect accepted commands and change its own phrase-selection rule. The seed, rule revisions and accepted input events make the digital performance traceable.

Scalp EEG can contain muscle, eye-movement and electrical artifacts. The decoder and artifact handling must establish what its outputs mean; the kernel cannot infer that from packed words. EMG can also be an intentional muscle-control input, but it should be named as such rather than described as brain reading. [S8]

First slice and fun risk. Keep two selectable actions, visual feedback and a conventional input fallback. First determine whether the participant experiences reliable agency. A wrong selection should be reversible in the digital composition. Full sensory neural output belongs to the separate research horizon H01, not to this input-only proposal.

E12 Flavor Worlds

Taste and scent entertainment | Custom installation and research hardware horizon

Explore a tiny virtual orchard, discover a scent clue and assemble a flavor-themed musical scene. Picking a virtual fruit selects a declared sensory event. The fruit's encoded recipe can inspect which variants the participant explicitly preferred and propose a different sequence for the next round.

Opening and loop. Begin with a visual orchard and authored sound. Follow a scent cue to one of two virtual plants, choose a variation and keep the scene you enjoy. A later, appropriately engineered installation could pair events with taste output. The puzzle is discovering combinations and composing a sequence, not consuming an unlimited stream of stimuli.

Wearable olfactory research has demonstrated programmable odor presentation, and the e-Taste study demonstrated a sensor-actuator interface for selected taste sensations. These are external research precedents, not atomOS capabilities or universal flavor reproduction. Device response, persistence, cleaning and participant-specific suitability constrain a real installation. [S9, S10]

Substrate and first slice. Klein connections organize the orchard's unusual paths. Log-polar regions and packed predicates select virtual interaction events. A versioned recipe contains the declared cross-sensory mappings. Start with the ordinary audiovisual scene and a small authored scent palette through an appropriate existing interface; investigate taste hardware as a separate project. A saved seed reproduces the declared content recipe, not the same taste experience in every person.

H01 Neural Sensation Theatre

Hypothetical entertainment application of bidirectional neural interfaces

Imagine exploring a virtual sculpture through a future, suitably validated neural interface that conveys a limited set of distinguishable tactile events. You choose one sensation motif, change its spatial rule and compare the next version. The engine could maintain the world state, propose bounded content variations and record the session recipe.

Clinical research has evoked tactile percepts and used them to improve robotic-arm control in a participant with tetraplegia. That supports a specific sensory-interface precedent, not a general capability to write complete worlds, emotions or pleasure into a brain. An entertainment adaptation would be a distinct research program. The cited FDA guidance concerns implanted BCI devices for paralysis or amputation. [S11, S12]

H02 Biochemical Experience Composition

Speculative horizon beyond ordinary taste and smell

One possible future research question is whether an experience could coordinate digital content with a separately validated biochemical interface to support a participant-chosen affective experience. The kernel could represent a model, compare explicitly reported preferences and track versioned proposals. The present work establishes no such body model, actuator mapping or reliable control of pleasure.

Chemical smell and taste act at sensory interfaces; directly controlling neurotransmitters, hormones or systemic drug effects is a different proposition. Encoding a desired response as an SDF or a LUT word does not make that response achievable. This branch is recorded as a hypothetical possibility space, without dosing, stimulation settings or a self-experimentation procedure.

Design rules for embodied self modification

The interesting self-referential loop includes the experience generator itself: it knows which rules produced a scene, can inspect selected outcomes and can propose changes to those rules. Connecting that loop to a body requires a separate, explicit measurement and output model. The participant is not interchangeable with the simulation state.

Keep content changes and device limits separate

A world may revise its melody, branching rule or next visual scene. That does not grant it permission to revise the physical room boundary, a recipient's chosen channels or a device's output limits. Put those constraints in a separate interface layer outside the application's self-editable rule set. This is a proposed design boundary, not a claim about a delivered hardware implementation.

For example, Sensory Loom may select a different supported phrase after an explicit preference input. It may not infer permission to increase physical output from a noisy pulse measurement. The player can pin the current content rule, lower their preference setting or end the session through controls the generated world cannot remove.

Distinguish three kinds of feedback

Declared user input includes an intentional button, gesture, confirmed decoded command or explicit preference. It can directly request a content change under the application's rules.

Measured physiological data includes a particular signal and its quality information. A separately validated mapping can make it a game input. Its meaning must not silently change from a measurement into an emotion, a biochemical state or consent.

Predicted response is a model output. Record it as a prediction and compare it with suitable observations. A canonical hash records the prediction accurately; it does not establish that the prediction is true.

Replay has a physical limit

The system can retain and restore digital state, rule versions and recorded input events. It cannot roll back a person's body or guarantee the same sensation when it replays an output sequence. Device response, calibration, context and the participant can differ. Any replay claim must specify whether it covers the logical content, emitted commands or a measured device response.

The core archive should retain content recipes and accepted edits. Raw physiological recordings require their own retention choice. A hash is not encryption or automatic anonymization. If data are omitted from a retained package, describe the resulting replay limit instead of implying the full session can be reconstructed.

Expansion order for this branch

First establish a readable XR view of a small Klein world. Add an ordinary audio or haptic channel and let the participant explicitly choose a variation. Next evaluate one measured input with confidence and missing-data handling. Explore novel taste, neural or biochemical interfaces only as distinct research efforts with their own evidence. This sequence lets the entertainment design develop while keeping claims attached to what has actually been implemented and observed.

A broader map of entertainment possibilities

The developed concepts cover puzzles, attachment, action, planning, expression, social creation and embodied sensory experiences. The following branches expand that map. Each is a proposal to investigate, with a concrete player action that could become a small playable experiment.

ID	Branch	Concrete entertainment mechanic
X01	Impossible racing	Plan a reflected seam shortcut and let a car's rule controller revise its next boost after a checkpoint.
X02	Billiards and pinball	Bank a ball across a seam; a bumper changes its next response after being hit.
X03	Living board game	Place pieces that carry inspectable move rules and can rewrite one rule after a capture.
X04	Dungeon horticulture	Grow a dungeon organism, then enter it to collect objects that alter its next growth cycle.
X05	Rule sculpting toy	Shape a kinetic sculpture by editing the small programs that generate its moving parts.
X06	Mirror ghost mystery	Compare retained versions of a room to find which machine changed its own rule and caused the disappearance.
X07	Cooperative escape room	One player navigates the Klein room while another edits a limited part of its rulebook.
X08	Creature tournaments	Prepare behavioral recipes, observe short matches and revise the training environment between rounds.
X09	Living comic	Choose objects inside a panel to alter the rule that generates the next panel's state; authored art and story logic remain required.
X10	Installation playground	Move through a local interactive display whose shapes inspect recent interactions and change their next response.

Axes for expanding the possibility space

Control: move, edit, arrange, breed or perform. **Self-edit scope:** an object, its generator or its edit policy. **Timing:** an event, turn, bar or paused session. **Persistence:** a favorite object, snapshot, family tree or replay. **Presentation:** flat chart, seam doors, linked views or immersion. These choices define substantially different experiences on the same required topology.

These ideas can also be implemented with other architectures. Their fit here comes from using represented rules, topology and revisions coherently; no exclusivity or automatic performance advantage is claimed.

The first playable and the expansion record

Foldhouse prototype outline

Build a complete E01 puzzle: move through a readable Klein room, change an instruction and observe one machine-authored edit. Completion opens a visible exit. This slice establishes whether the interaction is enjoyable and understandable.

Step	Playable result	Decision it supports
1	Walk, push an orb and cross a previewed reflected seam	Can the player predict movement?
2	Edit a two-slot instruction panel and solve the plate puzzle	Does editing produce a satisfying insight?
3	Let the machine inspect a plate event and propose one rule edit	Is the self-change understandable and useful?
4	Verify and commit that edit between ticks; show its effect	Does the player retain confidence in cause and effect?
5	Restore a retained version and replay recorded inputs	Can the player revisit and compare experiments?

The substrate path needs a full-digest seed projection, a rule-record format, feedback, a bounded edit interpreter and transactional publication. Define how objects map into words while preserving whole-word absorption; a per-object collision system cannot silently substitute different semantics. Presentation needs movement, a camera, seam cues, a rule editor, animation and sound. A fixed working set and explicit object caps keep the slice reviewable.

Cache behavior must be profiled under the actual game workload, including its scheduling and interference. Gameplay frame time must include orchestration, rendering and any verification overhead. The earlier kernel timing and residency studies are useful baselines, not a game frame-rate result.

What to observe in a play session

Observe whether a player predicts the reflected return, makes an intentional edit, recognizes the machine's self-edit and wants another arrangement. Note confusion without coaching. These are proposed observations, not completed playtests. Reduce overwhelming choices before expanding content.

Reusable concept entry

For each branch, record its ID, title, player fantasy, first minute, repeated loop and progression. Specify the Klein and log-polar role, predicate mapping, self-inspection data, permitted edits and triggers, seed recipe, retained history and memory policy.

Finish with the smallest playable version, its required systems, the main fun risk and a concrete observation that would justify expansion. Attach evidence separately as it becomes available: implementation status, measured performance and playtest findings. Keep the original proposal so later revisions remain traceable.

Version record: 0.1 includes twelve concepts, two speculative horizons and ten additional branches, including the spatial and biophysical extension.

Recorded architecture discussion

Conversation record, 13 September 2026. The user's architectural clarification and the following assistant reply are preserved below. Formatting is simplified for this document. The game concepts on the preceding pages expand this discussion into entertainment proposals.

User

yes but as the klein bottle log encoded polar LUT self referential engine packed as seeded SHA256 self referential and modifying applications

Assistant

You mean applications that can read and change their own encoded rulebooks, running on your SHA-256-seeded, Klein-bottle, log-polar LUT substrate. That is a more specific direction than an ordinary procedural engine.

ELI5: imagine a world made from programmable tiles. Its inhabitants can read the instructions that make the world work and propose edits to those instructions. The engine checks each proposed edit before accepting it.

In your architecture, the pieces have distinct jobs:

SHA-256 genesis: establishes identity and seeds placement. Your document maps the digest's two halves into log-radius and angle. A specified generator can also derive initial application data from that seed.

Klein topology: defines the world's actual connections, including the reflected seam crossing. It is part of the application's geometry and behavior.

Packed LUT words: encode operator predicates for texture-backed execution, with the working set kept as cache-resident as measurements support.

Self-reference: the application can inspect an encoded description of its own state and rules.

Self-modification: it proposes changed rules, verifies them, and commits a new immutable version between execution epochs.

Hash-linked history: records which accepted version came from which predecessor.

That identity/seed distinction, versioned configuration, and chained history are described in your formalization, pages 8, 25 and 30.

The strongest applications I would branch into are:

An executable world you can fork: A seeded world contains editable descriptions of its own machinery. Creatures or players change those descriptions, producing new behaviors and traceable descendant worlds. Pause, inspect, branch and replay retained versions.

A visual programming world: Build machines directly in the Klein domain. Their spatial arrangement and encoded rules form an executable program. A machine can inspect and rewrite its own connections through checked updates.

An evolving design laboratory: A generator creates patterns or mechanisms, measures the result, then modifies its own generating rules toward an explicit objective. Every accepted result retains its recipe and ancestry.

Discussion continuation and source record

Assistant reply continued

My first choice is the executable world: a small, inspectable universe where editing the world can also edit the program that produces its next state.

Two distinctions keep that ambition precise. A SHA-256 digest identifies content; it does not pack an arbitrary application into 256 recoverable bits. And bounded storage permits continued evolution through reuse, reclamation or external history - not unlimited simultaneously retained information.

The current kernel provides validated building blocks. The integrated SHA-256 seed mapping, self-description interface and closed rule-modification loop still need implementation. That is the concrete next architectural step toward the engine you are describing.

User request for this document

Yes but literal entertainment purposes beyond just conceptual stuff, do document the latest turn in a document to later be expanded as a broad overview of possibility spaces

Source record

S1. Tom Klootwijk, atomOS v3.6 Total Integrated Engine, M1, 43 pages. File:

atomOS_v3.6_Total_Integrated_Engine_Tom_Klootwijk.pdf. Page 8 separates full identity, seed projection and quantized occupancy. Page 25 specifies versioned configuration snapshots and commit semantics.

Page 30 specifies the ledger and conditional replay. SHA-256:

b51651c007c560775ff1950e278789cd7674e131ec71a09ebd9254cebe9c19ce.

S2. Final SDF and Klein evidence summary, 13 September 2026. Repository path:

results/sdf_klein_20260913/final_summary.json. Records the measured joint and bulk profiles, validation counts and scope boundaries. The separate distributed bulk profile retained up to 4.625 MiB in its measured schedule; this does not transfer that capacity to every game or kernel.

S3. Joint SDF and NOR controller with Klein lineage. Repository path: docs/SDF_JOINT.md. Specifies the actual one-way controller-to-geometry coupling, immutable texture reads, staged publication and finite resource limits. Mutable state, circuit wiring, identity records and traces remain outside the immutable operator texture.

S4. Source operator SDF audit. Repository path: docs/SOURCE_OPERATOR_SDF_AUDIT.md.

Distinguishes the original binary SDF_WORD rules from newly declared scalar geometry and records the missing definitions needed for a complete original operator scalar SDF bank.

S5. Concept attribution. Repository path: AUTHORSHIP.json. Tom Klootwijk is the engine concept author.

Entertainment pitches in this overview are AI-assisted design proposals developed in this conversation; they are not claimed as implemented features of the original formalization.

Interpretation rules for future expansion

Keep recurrence, self-description and self-modification separate. A program can recur without inspecting itself, and it can inspect itself without changing its rules. For every claimed self-modifying feature, identify the represented rule, the observation that triggers a proposal and the accepted before-and-after version.

Keep abstract computational expressiveness, native execution evidence and entertainment value separate. A mathematical simulation argument is not proof of arbitrary binary correctness, and a cache measurement does not establish that a game is fun. Add those evidence categories to each concept as work progresses.

Spatial sensory and neural sources

These primary sources establish external interface precedents and their scope. The proposed atomOS entertainment applications are design inferences developed here, not findings of these sources. Accessed 13 September 2026.

S6. Khronos OpenXR Working Group. OpenXR Spatial Entities Extensions Released for Developer Feedback, 10 June 2025. Describes spatial entities, plane and marker tracking, anchors and persistence, with implementation support to be checked on each runtime. [Read the Khronos announcement.](#)

S7. bHaptics developer documentation. Documents game integration through supported tools and SDKs. Establishes a programmable haptic interface category, not the feasibility of every proposed sensation. [Read the developer documentation.](#)

S8. OpenBCI documentation. Describes EEG, EMG and ECG measurement categories and noise considerations. Supports the distinction between an acquired signal, artifacts and a separately defined application input. [Read the biosensing overview](#) and [noise documentation](#).

S9. Liu and colleagues. Soft, miniaturized, wireless olfactory interface for virtual reality. Nature Communications 14, 2297, 2023. Reports wearable olfactory research devices and demonstrations. [Read the research article.](#)

S10. Chen and colleagues. A sensor-actuator-coupled gustatory interface chemically connecting virtual and real environments for remote tasting. Science Advances 11, eadr4797, 2025. Reports the e-Taste research interface and human perception experiments. [Read the research article.](#)

S11. Flesher and colleagues. A brain-computer interface that evokes tactile sensations improves robotic arm control. Science 372, 831-836, 2021. Reports a bidirectional interface used by a participant with tetraplegia for a specific motor and tactile task. [Read the author manuscript.](#)

S12. US Food and Drug Administration. Implanted Brain-Computer Interface Devices for Patients with Paralysis or Amputation, Non-clinical Testing and Clinical Considerations. Final guidance, 20 May 2021. Establishes the clinical device context of that guidance, not an entertainment endorsement. [Read the FDA guidance.](#)

Recorded scope extension

User

Yes but also real world spatial VR/AR/MR dare I say neural interface and biophysical entertainment and pleasure (sensory, biochemical, etc.) as hypothetical applications added as well

This extension is represented by E07-E12, H01-H02 and the embodied design rules. The scope includes pleasure as a legitimate entertainment goal while keeping the user's reported experience, sensor measurements and biological control claims distinct.